Further Local Rules for Distance-Only Shortest Paths: Supplement

Nikolai Khobotov
Independent researcher
nicholaask@gmail.com
ORCID: 0009-0005-3612-830X

October 2026

1 Setting

This supplement accompanies the paper *Exact Distance-Only Complexity of Small Topologies* [4], whose section "Local rules" measures a uniform program of four local rules; it defines five more rules, proves each correct, and gives their measured costs. A rooted topology is a directed graph with a source s and no weights. A program specialised to it adds and compares registers holding forms $\langle a, w\rangle$ in the arc weights $w \geq 0$, both operations charged, and Cai's $\mathrm{OPT}_{\mathrm{DIST}}(G, s)$ [2] is the least worst-case number of charged operations with which such a program holds every distance $d_w(s, v)$ (the paper's section "Model and benchmark"). In the *strict* game the adversary answers every comparison with $<$ or $>$, and only so that the cone of consistent weightings keeps a non-empty interior; the *path-register* strict game lets the program hold only forms coefficientwise below the form of a simple path from s, and its value V^{path} bounds $\mathrm{OPT}_{\mathrm{DIST}}$ from above. The paper's perturbation lemma turns a strategy that wins a strict game within K into a program correct on every $w \geq 0$, ties included, that makes at most K operations on every w. The *four-rule program* applies R0, dominator pruning, which deletes every arc $u{\to}v$ such that v dominates u, in linear time [1]; R1, which fixes a topological order of the strongly connected components of what remains and lets a selection range only over the labelled non-final vertices of the earliest component not yet finished; R2, local finality, which makes final without a selection a labelled non-final vertex whose in-neighbours kept by R0 are all final; and R3, array selection, which finds the least of h labels with $h - 1$ comparisons, and relaxes each out-arc of a vertex made final into a non-final vertex with one addition and, if its head is labelled, one comparison. It breaks ties by vertex number, and every label it holds is a simple-path form. A vertex is *labelled* once it holds a *label*, the length of a path from s to it; to *relax* an arc $u{\to}v$ out of a final u is to form $d(u) + w_{uv}$ and, if v is already labelled, to compare the two and keep the lesser; a vertex is *final* once the program has fixed its label as its distance; and the *candidates* of a selection are the vertices it ranges over. An *open cell* is the open set of weightings on which one fixed arborescence is the unique shortest-path tree. Its *cost* on a topology is its exact worst-case number of charged operations against the strict adversary, computed by walking its decision tree with `z3` over the weight cone and branching on every strict outcome the cone leaves feasible; an addition is charged when it creates a form not yet held.

Accountings. Three accountings of comparisons appear here. The paper charges *every comparison*, also one whose outcome the cone already decides, and states its costs in that

accounting only. With *decided comparisons free*, a comparison whose outcome the cone already decides is not charged; the program compiled from the decision tree omits it, which a uniform program could do only by testing the cone, a linear program. The *memo measure* of Section 8 makes a comparison free only when the comparisons made earlier on its branch imply its outcome. In this supplement a cost is taken with decided comparisons free unless its statement, its paragraph heading, its table caption or its section names another accounting; "both accountings" and "either accounting" mean decided comparisons free and every comparison charged. Which comparisons the cone decides depends on the order in which labels meet, so with decided comparisons free a cost is a cost on the arc set as numbered, and Section 8 measures costs on every numbering. The *value* of a topology is its path-register value V^{path}, and a program is *exact* on a topology when its cost equals the value. Every program here holds simple-path forms only, so its cost in each accounting is an upper bound on V^{path}, and the perturbation lemma applies to every cost.

The base set. The paper measures the four-rule program on 619 distinct decided topologies: every rooted topology with at most four vertices and at most seven arcs (62 rows), every one with five vertices and at most seven arcs (530 rows), and 42 further rows, 15 of them relabellings of rows of the first two tables, which bring 27 more topologies; we call them the *base set*, with $62 + 530 + 42 = 634$ rows. Among the further rows are the *glue* `4>1,0>2,1>2,2>1,2>3,0>4,3>4,4>3`, of value 8, two fed two-cycles joined both ways, and the *star* `0>1,0>2,0>3,1>4,2>4,3>4,4>1,4>2,4>3`, of value 9, three fed leaves around an unfed hub; the four-rule program is exact on the base set except on these two, where it is one above the value, in both accountings (the paper's subsection "Two losses at five vertices").

Names. The further rules are R4 to R8. The program $\Pi_{0\text{–}j}$ has the rules R0 to Rj, so the four-rule program is $\Pi_{0\text{–}3}$, and $\Pi_{0\text{–}6,8}$ has R0 to R6 and R8; no program combining R7 and R8 is evaluated. The evaluator, `theory/game/rule_programs.py` in the repository `https://github.com/primaryaesthetics/distance-only-topologies`, and the claim ledger `docs/ledger/CLAIMS.md` name rules and programs as in the second and third columns of Table 1.

Fallback. For R6, and so for $\Pi_{0\text{–}6}$, $\Pi_{0\text{–}7}$ and $\Pi_{0\text{–}6,8}$, the proof covers the least candidate of a selection but not that a selection has a candidate. In every program here, a selection with no candidate would range over every labelled non-final vertex of the current component; this *fallback* keeps the program correct, by the argument of Lemma S3.1 with no vertex left out, and its effect on the cost is not bounded. The evaluator flags every fallback, and none occurred on the tables or on the random draws evaluated.

What is held out. Each of R4 to R8 takes its mechanism from a topology on which the program without it loses: R4 from the glue and R5 from the star, R6 from three topologies of the five-vertex, eight-arc table of Section 2, R7 from the fed bidirected path, and R8 from the one loss of the six-vertex table. On those topologies a rule attains the value by construction. Every rule was fixed with the costs on the base set known, the rules from R6 on also with those on the five-vertex, eight-arc table and R8 with those on the six-vertex table, so exactness on their other rows shows that a rule breaks nothing there, not that it carries over. Held out from a rule are the topologies on which no program had been evaluated when the rule was fixed; the costs in this list are taken with decided comparisons free:

here	ledger	`rule_programs.py`
R0 to R3	the rules of the four-rule program	`four-rule`
R4, out-final exclusion	out-final exclusion	`five-rule`
R5, the hub	hub rule	`six-rule`
R6, the relay	relay	`seven-rule`
R7, the pass	piece pass	`eight-rule`
R8, the order of a selection	the nine-rule program's selection	`nine-rule`
array-selection Dijkstra	array-selection Dijkstra	`dijkstra`
local-final	local-final rule	`local-final`
pooled variant	pooled variant	`pooled`
four-rule program, $\Pi_{0\text{–}3}$	four-rule program	`four-rule`
$\Pi_{0\text{–}4}$	five-rule program	`five-rule`
$\Pi_{0\text{–}5}$	six-rule program	`six-rule`
$\Pi_{0\text{–}6}$	seven-rule program	`seven-rule`
$\Pi_{0\text{–}7}$	eight-rule program	`eight-rule`, with `--eight-rule`
$\Pi_{0\text{–}6,8}$	nine-rule program	`nine-rule`, with `--nine-rule`
two sweeps on P_k	two-sweep program	`two-sweep`, with `--bipath`
decided comparisons free	(default)	no flag
every comparison charged	every comparison charged	`--charge-decided`
memo measure	memo measure	`--charge-memo`

Table 1: Names of rules, programs and accountings here, in the claim ledger, and in the evaluator. The ledger names a rule by what it does; the evaluator's comments call the relay R7, which is R6 here. On the tables and with `--specials`, without `--eight-rule` or `--nine-rule`, the evaluator prints the programs from `dijkstra` to `seven-rule` side by side, and with either flag it prints `seven-rule` beside the named program; other inputs print the programs their help text names. `--random-seven` compares `six-rule` with `seven-rule` on random draws, and `--pieces` evaluates the two passes of R7 on abstract pieces.

- the five-vertex table for R4 and R5: R4 saves an operation on four rows, one of them a relabelling of the glue, each time reaching the value, and R5 changes no cost;
- the decided rows of the six-vertex table for R4 to R6: R4 and R5 change no cost, and R6 saves an operation on one row, reaching the value;
- random arc sets, of unknown value, on which the programs can only be compared with each other; no rule makes a program dearer on any of them. R5 is measured on the draws of R4, and every later rule on draws of its own, with their own seeds; a count is of the distinct arc sets in a draw, so two draws of the same shape, such as the 2993 and the 2992 distinct arc sets on six vertices below, are different samples;
- seven random topologies whose value was settled after a rule had saved an operation on them, all at the cost of the program with that rule: four for R6 and three for R8, with a fourth for R8 left between two values.

2 Tables and exactness

Besides the base set, the programs are evaluated on two tables played in the path-register game at one hour a row; only that game was played on them, so their values are values of V^{path}.

The *five-vertex, eight-arc table* holds the 426 rooted topologies on five vertices with eight arcs and a cycle, counted up to relabelling of the non-source vertices, out of 453 with eight arcs. The solver decides 400, with values from 3 to 10, and leaves 26 with lower bounds 8, 9 or 10. Seven of the 400 are relabellings of topologies of the base set, among them C_4 and the glue, each at its value there; the other 393 bring the decided topologies to 1012. On 22 of the 26 the least cost of $\Pi_{0\text{–}3}$, $\Pi_{0\text{–}4}$ and $\Pi_{0\text{–}5}$, in either accounting, meets the solver's lower bound, on three of them after a four-hour run, which decides V^{path} there; four stay open by one. So V^{path} is known on 1034 distinct topologies.

The *six-vertex table* holds 300 random arc sets on six vertices with seven to nine arcs and a directed cycle. The solver decides 288, with values from 2 to 10, and leaves 12 with lower bounds 9 and 10; on 8 of the 12 the least cost of $\Pi_{0\text{–}3}$ to $\Pi_{0\text{–}6}$, in either accounting, meets the bound, so V^{path} is known on 296 rows, and four stay open by one after four hours each.

decided topologies	$\Pi_{0\text{–}3}$	$\Pi_{0\text{–}4}$	$\Pi_{0\text{–}5}$	$\Pi_{0\text{–}6}$	$\Pi_{0\text{–}6,8}$
base set, 619 distinct	617	618	619	619	619
five vertices, eight arcs, 400 decided	393	397	397	400	400
six-vertex table, 288 decided	286	286	286	287	288

Table 2: The number of decided topologies on which each program is exact, as numbered, with decided comparisons free. On the rows of the second and third lines that the solver leaves undecided and on which a program's cost meets its lower bound, $\Pi_{0\text{–}5}$, $\Pi_{0\text{–}6}$ and $\Pi_{0\text{–}6,8}$ cost the value on all 22 of the five-vertex table, and $\Pi_{0\text{–}6}$ and $\Pi_{0\text{–}6,8}$ on all 8 of the six-vertex table.

In all, as numbered and with decided comparisons free, $\Pi_{0\text{–}6}$ is exact on all 1034 topologies with a known value of the base set and the five-vertex table, and $\Pi_{0\text{–}5}$ on 1031; on the 296 rows of the six-vertex table with a known value $\Pi_{0\text{–}6}$ is exact on 295 and $\Pi_{0\text{–}6,8}$ on all 296. The losses from which R6 and R8 take their mechanisms are these. On the five-vertex table $\Pi_{0\text{–}3}$, $\Pi_{0\text{–}4}$ and $\Pi_{0\text{–}5}$ cost 8 against the value 7, in both accountings, on

`0>1,0>2,1>3,2>4,3>1,3>2,3>4,4>3`;

`0>1,0>2,1>3,2>3,3>1,3>4,4>1,4>2`;

`0>1,0>2,1>3,2>3,3>1,3>2,3>4,4>1`.

In each a fed vertex u has a single out-arc, into vertex 3, and the optimal strategy opens with the addition $w(0{\to}u) + w(u{\to}3)$, before any comparison, then compares the other fed vertex's source arc with it. That comparison is a selection between the other fed vertex and 3 which leaves u out; where the formed sum is the lesser, it settles 3 and u together. On the six-vertex table $\Pi_{0\text{–}3}$ to $\Pi_{0\text{–}6}$ cost 10 on every numbering of `0>1,0>3,0>4,1>5,3>2,3>4,4>5,5>1,5>3`, of value 9, and $\Pi_{0\text{–}6}$ also with every comparison charged. Vertices 1, 3 and 4 are fed, and 2 is a sink fed only by 3. After its first selection the optimal strategy leaves 3 out of the second in two ways. Where $w(0{\to}4)$ is least it compares $w(0{\to}1)$ with $w(0{\to}4) + w(4{\to}5)$ alone: the arcs of 3 enter the final vertex 4 and the sink 2, which lies in a later component, so 3 can lower no label of the current component. Where $w(0{\to}1) < w(0{\to}4) < w(0{\to}3)$ it compares $w(0{\to}4)$ with $w(0{\to}1) + w(1{\to}5)$ alone, because its opening comparison already put $w(0{\to}3)$ above $w(0{\to}4)$.

Every comparison charged. With every comparison charged, $\Pi_{0\text{–}6}$ and $\Pi_{0\text{–}7}$ cost the value on every topology of the base set, and $\Pi_{0\text{–}3}$ to $\Pi_{0\text{–}6}$ on every one with at most five vertices and at most seven arcs. On the five-vertex table $\Pi_{0\text{–}7}$ costs what $\Pi_{0\text{–}6}$ costs, the value on 421 of the 422 rows with a known value, and on the six-vertex table the value on 295 of the 296; the exceptions are the topology of Computed fact S8.1 and the six-vertex loss above.

3 R4, out-final exclusion

R4. A labelled non-final vertex all of whose out-arcs kept by R0 enter final vertices takes no part in a selection; R2 settles it once its kept in-neighbours are final.

Which vertices R4 leaves out depends on the topology and the final set only, so the program is still defined by the topology alone.

Lemma S3.1. *On every weighting of an open cell, in* $\Pi_{0\text{–}4}$*, the least candidate of a selection has its distance as its label, and a selection always has a candidate.*

Proof. Assume that every final vertex holds its distance and has relaxed its kept out-arcs into non-final vertices; labels are lengths of paths, so no label is below a distance. Let y be a non-final vertex of the current component, P a shortest path to y, and x the vertex after the last final vertex of P. Every arc of P is kept, since R0 deletes no arc of a simple path, so x holds the label $d(x)$. The vertex x is not final and reaches y, so by R1 it lies in the current component. If $x \neq y$, its successor on P is not final, so R4 does not leave x out, and x is a candidate with label at most $d(y)$. Taking for y the selected vertex, its label is at most that of x if $x \neq y$, and is $d(x)$ if $x = y$; either way it is $d(y)$. Suppose a selection had no candidate. Then $x = y$ for every non-final y of the current component, so every such y is labelled and left out. A kept in-neighbour y' of such a y that is not final lies in the current component, since it reaches y, and has a kept arc into the non-final y, so it is not left out, which is impossible. So every kept in-neighbour of y is final, and R2, which is tried before a selection, settles y first. □

On the 634 rows of the base set R4 leaves a vertex out on the glue alone, where it costs 8, the value; elsewhere $\Pi_{0\text{–}4}$ makes the four-rule program's decisions. It misses the star: a leaf has all its out-arcs into final vertices only once the hub is final, the hub only once every leaf is, and in both cases R2 settles the vertex first. On the five-vertex table, with decided comparisons free, R4 leaves a vertex out on 5 rows and saves an operation on 4, each of value 8, one of them a relabelling of the glue; on the decided rows of the six-vertex table it leaves no vertex out. R4 leaves a vertex out on 48 of 397 distinct random arc sets on five vertices with 8 to 11 arcs and on 6 of 1000 on six vertices with 7 to 9 arcs; with decided comparisons free, $\Pi_{0\text{–}4}$ is cheaper than the four-rule program by one on 19 of the first and on 2 of the second, and dearer on none. Up to relabelling these 21 arc sets are 18 topologies, none of them a relabelling of a topology of the base set. One is a relabelling of a row of the five-vertex table: `0>2,0>4,1>3,1>4,2>1,2>3,3>2,4>1`, at 8 against 9, whose value there is 8. On the fed bidirected path P_k, with arcs $s \to i$ for every i and $i \to i+1$, $i+1 \to i$, every out-neighbour of a non-source vertex is also an in-neighbour, so a vertex R4 would leave out already has all its in-neighbours final and R2 takes it first: $\Pi_{0\text{–}4}$ is the four-rule program there.

4 R5, the hub

Call a vertex x *h-exact* when it is labelled and every in-neighbour of x kept by R0, other than h, is final.

R5. Tried after R2 and before a selection. If a non-final vertex h of the current component has no final in-neighbour and every in-neighbour of h kept by R0 is h-exact, each of them is relaxed into h and h is made final with no selection. A vertex made final does not relax into the out-neighbours its own form already runs through.

A kept in-neighbour x of h has a kept in-neighbour other than h: otherwise every simple path to x would end with $h \to x$, h would dominate x, and R0 would have deleted $x \to h$. So if those in-neighbours are final, x is labelled. R5 is correct: let x be the vertex before h on a shortest path to h. The part of the path up to x is a shortest path to x that avoids h, and the vertex before x on it is a kept in-neighbour other than h, hence final and already relaxed into x; so the label of x is $d(x)$, and the least candidate is $d(h)$. The last sentence of R5 changes nothing before a hub is made final, since until then every vertex a form runs through is final and arcs into final vertices are not relaxed. The condition that h have no final in-neighbour is not needed for correctness; without it the rule can take the place of a cheaper selection.

With decided comparisons free, R5 applies on 7 of the 634 rows of the base set and changes the cost only on the star, where it costs 9, the value; there it is the tree pass of the paper's subsection "Two losses at five vertices". On the five-vertex table it applies on 13 rows and on the decided rows of the six-vertex table on four, changing no cost. On the random arc sets above it applies on 6 and on 7 and costs what $\Pi_{0\text{–}4}$ costs on every one. A family where it matters is the bidirected star with j fed leaves and an unfed hub h, arcs $s \to i$, $i \to h$ and $h \to i$ for $i = 1, \ldots, j$ (Table 3). With decided comparisons free, $\Pi_{0\text{–}5}$ costs $4j - 3$: j additions and $j - 1$ comparisons for the hub, then one addition and one comparison for each leaf other than the one the hub's form runs through. The values for $j \geq 4$ are not known. On P_k the source is a final in-neighbour of every vertex, so R5 never applies there.

j	2	3	4	5
$\Pi_{0\text{–}3}$ and $\Pi_{0\text{–}4}$	5	10	16	23
$\Pi_{0\text{–}5}$ and $\Pi_{0\text{–}6}$	5	9	13	17

Table 3: Costs on the bidirected star with j fed leaves and an unfed hub, with decided comparisons free.

5 R6, the relay

Call an arc $u \to v$ of a labelled non-final vertex u *useful* if it is kept by R0 and v is neither final nor on u's form. At a selection, after R2 and R5 have had their turn, a candidate u is *eligible* for a relay if it has exactly one useful arc $u \to v$, v is unlabelled and in the current component, and every path among non-final vertices from another labelled vertex into u passes through v. To relay u is to relax $u \to v$ and leave u out of selections while its label stands; if the label of u later drops, u is a candidate again. A relay is made at most once between two finalisations. When a vertex v is selected, the vertices on its form whose labels are prefixes of that form are made final with it, and R5 may take a labelled vertex as its hub.

R6. Relay an eligible u when every other candidate x meets one of two clauses, where in_y is the number of kept arcs into y from non-final vertices and a_x the number of useful arcs of x:

H once x is final and its arcs relaxed, no labelled vertex has all its kept in-neighbours final, and v is the hub R5 takes next;

D $2(\mathrm{in}_x - \mathrm{in}_u) \;\geq\; a_x + [x \text{ has a useful arc into } v]$.

The clauses decide when a relay is made, not whether the program is correct. The proof of Lemma S3.1 extends as follows. With y, P and x as there, let z run along P from x; z holds the label $d(z)$, and if $z \neq y$, its successor z^+ on P is not final, lies in the current component, and is not on the form of z, since a vertex on that form is nearer than z and z^+ is farther. So z is not left out by R4, and if z is relayed with its current label, $z{\to}z^+$ is its one useful arc, the relay relaxed it with the label $d(z)$, and z^+ holds $d(z^+)$. Following relayed vertices along P reaches a candidate or y with its distance as its label, so the least candidate label is a distance. A vertex made final as a prefix of a selected vertex's form holds a prefix of a shortest path. With h labelled, R5's argument is unchanged, since the label h holds is the length of a path through one of its in-neighbours. The eligibility condition is not used; it limits relays to vertices whose label only a path through v can lower. Whether a selection of $\Pi_{0\text{–}6}$ can be left without a candidate is not proved; the evaluator's flag is raised on none of the topologies below.

Clause H has an exchange argument for the opening. Where u's path to v is the least label, the relay settles u and v with one selection where $\Pi_{0\text{–}5}$ makes at most two. Where another candidate x is selected first, the next step of $\Pi_{0\text{–}5}$ under H is the hub rule on v, which forms $w(u) + w(u{\to}v)$ itself; where it selects u first and x before v, finalising u forms the same sum. Either way the relay's addition is one $\Pi_{0\text{–}5}$ makes as well. The argument does not bound what follows. Clause D has no argument: it weighs the relaxations into a heavy, still unsettled x that $\Pi_{0\text{–}5}$ pays before x is locally final against the useful arcs of x that the relay pays for when x wins, as if there were two candidates and the rest cost the same on both sides. It is a heuristic, and its effect is known only on the topologies listed here.

With decided comparisons free, R6 relays on none of the rows of the base set. On the five-vertex table it relays on 4 rows, all at the first selection: by H on the first of the three losses above and on one row where $\Pi_{0\text{–}5}$ already attains the value, and by D on the other two losses; on the three losses it costs 7, the value, and on every other row, the 26 undecided ones among them, what $\Pi_{0\text{–}5}$ costs. On the six-vertex table it relays on one row, by D, where it costs the value, 8, one below $\Pi_{0\text{–}5}$. On 1435 distinct random arc sets on five vertices with 9 to 11 arcs and 2993 on six vertices with 7 to 9 arcs, with decided comparisons free, R6 relays on four, by D on the one with five vertices and by H, H and D on the three with six; on each $\Pi_{0\text{–}6}$ is one below $\Pi_{0\text{–}5}$, and on the others the two cost the same. The solver decides all four at the cost of $\Pi_{0\text{–}6}$, 7, 9, 8 and 8, and on each the optimal strategy it prints makes the relay's addition before any comparison, on three of them as its first operation. With every comparison charged, $\Pi_{0\text{–}6}$ costs 7, 12, 18 and 25 on P_3 to P_6, what the four-rule program costs there with decided comparisons free.

6 R7, the pass

The non-final vertices of the current component fall into *pieces*, the components of the undirected graph of the kept arcs among them. A piece is eligible when it is a tree on at least two vertices and the form of each of its labelled vertices runs through final vertices only, besides the vertex itself.

R7. Tried after R5 and before R6. On each eligible piece, taken as an abstract topology with a source arc into each labelled vertex, compare the cost of $\Pi_{0\text{–}6}$ with that of two passes from a root, every arc from child to parent bottom-up and then every arc from parent to child top-down, both with every comparison charged. The first eligible piece on which the passes from the cheapest root cost strictly less is settled by them.

Both costs are taken with every comparison charged because with decided comparisons free a cost depends on the numbering (Section 8). The test is an exact worst-case evaluation on the piece, exponential in its size; it reads the topology and the final set only, so it is free control in Cai's model, but unlike R0 and R1 it is not linear-time. R7 is correct: every kept in-neighbour of a piece vertex outside the piece is final and relaxed, so on a shortest path to a piece vertex the vertex after the last vertex outside the piece holds its distance, and the rest of the path is the tree path; the bottom-up pass carries the least label to the root and the top-down pass carries it out.

All results for R7 are with every comparison charged. R7 never applies on the rows of the base set, of the five-vertex table or of the six-vertex table, so $\Pi_{0\text{–}7}$ costs there what $\Pi_{0\text{–}6}$ costs. On P_k the whole path is one piece at the start, and $\Pi_{0\text{–}7}$ costs 7, 12, 16 and 20 for $k = 3, \ldots, 6$ against 7, 12, 18 and 25 for $\Pi_{0\text{–}6}$; from $k = 5$ the passes apply and settle the path at $4k - 4$, the two-sweep cost. Over 258 abstract pieces up to relabelling, every tree on two to four vertices with each edge one way or both and every bidirected tree on five, each with a nonempty fed set and every vertex reachable from the source, the passes are cheaper on 15. The one on four vertices is the bidirected path 2-1-4-3 fed at 1, 2 and 3, at 9 against 10; its value is 9, and $\Pi_{0\text{–}7}$ costs 9 on every one of its 24 numberings. On 2962 random arc sets on five and six vertices $\Pi_{0\text{–}7}$ is never dearer than $\Pi_{0\text{–}6}$ and cheaper on four. On the 183 fed bidirected trees on six non-source vertices up to relabelling, with both arcs of every tree edge and source arcs into a nonempty set, it is never dearer and is cheaper on 82, by 1 to 6; it never saves when one or two vertices are fed, and the largest saving, 17 against 23, is on eight trees fed at five vertices. The values of these trees are not known.

7 R8, the order of a selection

R8. Three changes to the selection of $\Pi_{0\text{–}6}$. R4 reads only the kept out-arcs into the current component. A candidate whose label is known to be greater than another candidate's is left out. The candidates are scanned in increasing order of the number of labelled vertices of the current component that the exclusion would keep once that candidate is final and its out-arcs relaxed, least vertex first among equals, so that when the last one scanned wins, the least of the others is already known.

Known means decided by the cone, which takes a linear program to check; under the memo measure it means implied by the comparisons made earlier on the branch, and with every comparison charged no candidate is left out as a known loser. Any scan order returns the least candidate, and a known loser is not the least, so only the restricted exclusion needs an argument. It is the argument of Lemma S3.1 and of R6 with one more line: on a shortest path to y, the successor of a non-final vertex of the path that is not y is non-final and reaches y, so it lies in the current component, and the arc into it is a kept arc into the current component. The restricted exclusion is the first move of the six-vertex loss: there the arcs of 3 enter only the final vertex 4 and the sink 2, which lies in a later component. The ordered scan reproduces the second, the

reuse of a comparison an earlier selection made, which the earlier programs get, with decided comparisons free, only where the cone decides it and so only on some numberings.

With decided comparisons free, $\Pi_{0\text{–}6,8}$ is exact on every topology of the base set and on every row of the five-vertex and six-vertex tables with a known value, the six-vertex loss included. On 4192 random arc sets with six and seven vertices it is never dearer than $\Pi_{0\text{–}6}$ and is cheaper on 21. Of the four of them with at most nine arcs, the solver decides two at the cost of $\Pi_{0\text{–}6,8}$, its lower bound meets that cost on a third, and the fourth lies between 9 and 10.

8 Numberings and accountings

The rules break ties by vertex number: R2 and R5 take the least eligible vertex, R3 compares labels in the order of their vertices, and R6 tries candidates in a fixed order. The value of a topology does not depend on the numbering. A program's cost can, through the comparisons the cone decides for free. $\Pi_{0\text{–}3}$ to $\Pi_{0\text{–}6}$ were evaluated on every numbering of the non-source vertices of each of the 1034 topologies with a known value of the base set and the five-vertex table, except the eight fed cycles with seven to nine vertices, which were evaluated on 162 to 300 drawn numberings each.

Computed fact S8.1. With decided comparisons free, on every numbering evaluated of each of those 1034 topologies the four-rule program costs what it costs as numbered. So do $\Pi_{0\text{–}4}$, $\Pi_{0\text{–}5}$ and $\Pi_{0\text{–}6}$, on each topology but one, `0>1,0>2,0>3,1>2,2>4,3>4,4>1,4>3`, of value 8: there they cost 8 on 4 of its 24 numberings, the tabulated one among them, and 9 on the other 20, and with every comparison charged 9 on all 24. Counted on every numbering, $\Pi_{0\text{–}6}$ is exact on 1033 of the 1034 and $\Pi_{0\text{–}5}$ on 1030. $\Pi_{0\text{–}6,8}$ costs the same on every numbering evaluated of each of these topologies and of every row of the six-vertex table, and on each with a known value it costs the value, the exception above included.

The exception is one of the four topologies of the five-vertex table on which R4 saves an operation. There the saving is a comparison the cone decides for free on 4 of its 24 numberings, the tabulated one among them, and on the other 20 there is no saving. Of the three topologies other than the glue on which R4 saves, it saves on every numbering of two; with every comparison charged, $\Pi_{0\text{–}4}$ to $\Pi_{0\text{–}6}$ cost the value on every numbering of those two and of the glue. The savings of $\Pi_{0\text{–}6}$ over $\Pi_{0\text{–}5}$ do not depend on the numbering: on the three losses of the five-vertex table and the four random topologies decided at its cost, it costs the value and $\Pi_{0\text{–}3}$ to $\Pi_{0\text{–}5}$ one more on every numbering, with decided comparisons charged or not. Outside the tables the order matters more. On the bidirected path 2-1-4-3 fed at 1, 2 and 3, of value 9, $\Pi_{0\text{–}3}$ to $\Pi_{0\text{–}6}$ cost 9 on 6 of its 24 numberings and 10 on the other 18, and 10 on all of them with every comparison charged. $\Pi_{0\text{–}7}$ costs 9 on all 24 with every comparison charged, and $\Pi_{0\text{–}6,8}$ 9 on all 24 with decided comparisons free.

Every comparison charged. With every comparison charged, $\Pi_{0\text{–}6}$ costs the value on every numbering of 421 of the 422 rows of the five-vertex table with a known value, and $\Pi_{0\text{–}4}$ and $\Pi_{0\text{–}5}$ on 418; the exception is the topology of Computed fact S8.1. On the six-vertex loss $\Pi_{0\text{–}6,8}$ and $\Pi_{0\text{–}6}$ both cost 10 on every numbering, since no candidate is left out as a known loser.

The memo measure. Under the memo measure a comparison is free only when the comparisons made earlier on its branch imply its outcome, by transitivity and because a form is less than

every form containing it; every other comparison is charged. $\Pi_{0\text{–}6}$ and $\Pi_{0\text{–}6,8}$ cost under it what they cost with decided comparisons free on every topology of the base set, on the five-vertex and six-vertex tables as numbered, on every numbering of the six-vertex loss, and on 2992 random arc sets on six vertices. On the five-vertex table this includes 8, the value, on the exception of Computed fact S8.1 as numbered; its other 23 numberings were not evaluated under this measure.

9 Details of the four-rule program

Table 4 lists the five topologies with four vertices on which local-final is not exact, with the rule that closes the gap, R0 on the three where vertex 1 dominates vertex 3, and R1 on the two where vertex 3 lies downstream of the two-cycle $\{1, 2\}$. Their costs are the same in both accountings.

topology	V^{path}	Dijkstra	local-final	four-rule	rule
`0>1,0>2,1>2,1>3,3>1`	3	5	4	3	R0
`0>1,0>2,1>3,3>1,3>2`	3	5	4	3	R0
`0>1,0>2,1>2,1>3,3>1,3>2`	5	7	6	5	R0
`0>1,0>2,0>3,1>2,1>3,2>1`	5	7	6	5	R1
`0>1,0>2,0>3,1>2,1>3,2>1,2>3`	7	9	8	7	R1

Table 4: The five topologies with four vertices on which local-final is not exact, with their costs and the rule of the four-rule program that closes the gap.

The paper states the costs of the four-rule program with every comparison charged. With decided comparisons free its counts on the base set are the same: it is exact on the 62 rows with at most four vertices, on the 530 with five vertices and at most seven arcs, and on 40 of the 42 further rows, one above on the glue and the star; array-selection Dijkstra is exact on 18, 47 and 7 of these and local-final on 57, 434 and 35. Two histograms differ between the accountings: on five vertices local-final is above the value by 1 on 87 rows and by 2 on 9, against 86 and 10 with every comparison charged, and array-selection Dijkstra is above it by 1 to 6 on 82, 120, 132, 92, 42 and 15 rows, against 82, 119, 127, 90, 50 and 15. On the fed bidirected path P_k, $k = 3, \ldots, 7$, the two sweeps cost 8, 12, 16, 20 and 24 and the four-rule program 7, 12, 18, 25 and 33 with decided comparisons free, over 22, 134, 986, 8558 and 84506 leaves, and the same with every comparison charged, the costs the paper gives. The path-register search on P_4 refutes every budget through 8 within 36000 seconds. The glue is, up to relabelling, a row of the five-vertex, eight-arc table.

At scale. The run on concrete weights charges a selection over the h labelled non-final vertices of the current component $\lceil \log_2(h+1) \rceil$ comparisons, as a heap would be charged, in place of the $h-1$ of R3. On the road network `USA-road-d.NY` [3] from its vertex 1 the four-rule program makes 196701 selections, prunes 54682 arcs and finds 53480 components, at 2.92 charged operations per arc, against 3.21 for local-final alone. Alternating passes from the least entry converge in 4 to 9 passes on the fed grid and in 4 to 10 on the random digraph, and on `USA-road-d.NY`, where they do not converge within 64 passes, the order reversals on shortest-path-tree paths reach 639, with mean 265.64, in depth-first order and 300, with mean 98.62, in breadth-first order.

10 Where each result is recorded

Every number here comes from a recorded run in the repository `https://github.com/primaryaesthetics/distance-only-topologies`: a directory under `experiments/` holding the standard output and a manifest with the exact command, and a row of the claim ledger that states the number with that directory. The evaluator is `theory/game/rule_programs.py` and the run on concrete weights `ref/rule_programs/scale_probe.py`.
Representative commands:

```
python theory/game/rule_programs.py --n5 --charge-decided
python theory/game/rule_programs.py --random-seven 6,7,9,3000,9102 --jobs 4
python theory/game/rule_programs.py --pieces 5,5 --jobs 3
python theory/game/rule_programs.py --eight-rule --charge-decided \
       --bipath 3,4,5,6
python theory/game/rule_programs.py --nine-rule --jobs 4 \
       --random 6,7,9,3000,9501 --charge-memo
```

A table run reads the standard output of the solver runs with `--table FILE,FILE,...`, and `--numberings` evaluates every numbering. The directories under `experiments/`, a trailing star standing for several with that prefix:

- the base set: `2026-10-14-rule-programs-n4`, `2026-10-14-rule-programs-n5`, `2026-10-14-rule-programs-specials`, `2026-10-15-rule-programs-specials-*`, `2026-10-17-rule-programs-n4-charged`, `2026-10-17-rule-programs-n5-charged`;
- the values of the glue and the star: `2026-10-14-glue-c2-c2-replace-path`, `2026-10-15-glue-c2-c2-replace-generic`, `2026-10-15-star-unfed-hub-path`, `2026-10-15-star-unfed-hub-generic`;
- corner and envelope on the two larger tables: `2026-10-17-table-bounds-n5m8-n6`;
- R4: `2026-10-15-rule-programs-five-rule-*`;
- R5: `2026-10-15-rule-programs-six-rule-*`;
- R6: `2026-10-16-rule-programs-seven-rule-*`, `2026-10-17-rule-programs-seven-rule-n6`, `2026-10-16-seven-rule-cheaper-solve-*`;
- R7: `2026-10-17-rule-programs-eight-rule-*`, `2026-10-17-pass-row-solve-*`;
- R8: `2026-10-17-rule-programs-nine-rule-*`, `2026-10-17-n6-nineb-solve-*`;
- numberings: `2026-10-16-rule-programs-numberings-*`, `2026-10-17-rule-programs-numberings-*`;
- the five-vertex, eight-arc table: `2026-10-15-n5m8-cyclic-chunk*`, `2026-10-16-n5m8-*`, `2026-10-16-opt-dist-census-n5m8`, `2026-10-16-rule-programs-six-rule-n5m8`;
- the six-vertex table: `2026-10-16-n6-cyclic-random-chunk*`, `2026-10-17-n6-*`;
- the fed bidirected path: `2026-10-14-rule-programs-bipath`, `2026-10-15-rule-programs-bipath-to-6`, `2026-10-19-rule-programs-bipath-charged`, `2026-10-15-fed-bidirected-p4-path`;
- concrete weights: `2026-10-15-rule-programs-scale-live-heap`.

11 Open problems

1. A program with both R7 and R8: $\Pi_{0\text{–}6,8}$ has no pass, and $\Pi_{0\text{–}7}$ does not order its selections. And the values of the fed bidirected trees on which R7 saves, by up to six operations on six non-source vertices.

2. The value of the bidirected star with $j \geq 4$ fed leaves, which $\Pi_{0\text{–}5}$ bounds by $4j - 3$.

3. An exchange argument for clause D of R6, or a topology on which the relay D licenses is dearer than selecting; and the topologies whose values are open by one: four of the five-vertex, eight-arc table and four of the six-vertex table.

4. Topologies with known values on which R5 changes the cost. It changes it only on the star; R4 saves on three topologies other than the glue, two of them on every numbering with every comparison charged, and the evidence that neither rule ever costs more comes from topologies whose values are not known.

5. $V^{\mathrm{path}}(P_4)$, which lies between 9 and 12, and the values of P_5 to P_7.

References

[1] Stephen Alstrup, Dov Harel, Peter W. Lauridsen, and Mikkel Thorup. Dominators in linear time. *SIAM Journal on Computing*, 28(6):2117–2132, 1999.

[2] Bin Cai. Beyond distance ordering: resource complexity and universal optimality of exact labeled directed shortest paths. arXiv:2609.04825, 2026.

[3] Camil Demetrescu, Andrew V. Goldberg, and David S. Johnson, editors. *The Shortest Path Problem: Ninth DIMACS Implementation Challenge*, volume 74 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, 2009.

[4] Nikolai Khobotov. Exact Distance-Only Complexity of Small Topologies. 2026. Repository `https://github.com/primaryaesthetics/distance-only-topologies`.